

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Алгоритмы ориентации в пространстве

Отчёт по лабораторной работе № 1

Работу
выполнил:
Сухов Н. М.
Группа: R3335
Преподаватель:
Каплин А. В.

Санкт-Петербург
2026

Содержание

1. Цель и задачи лабораторной работы	3
2. Описание программной среды и модели	3
3. Краткая теоретическая часть	4
4. Основы сцены и управление объектами	4
5. Визуализация и преобразования систем координат	7
6. Прямая кинематика (FK)	8
7. Обратная кинематика (IK)	10
8. Управление движением (позиция/скорость)	13
9. Коллизии и простое избегающее поведение	17
10.Сенсоры	21
11.Выводы	26

1. Цель и задачи лабораторной работы

Целью лабораторной работы является изучение базовых приёмов работы с объектами и манипуляторами в среде CoppeliaSim, а также получение практических навыков программного управления сценой с использованием встроенных Lua-скриптов. В работе рассматриваются способы задания положения объектов, преобразования между системами координат, расчёт прямой и обратной кинематики, построение простых траекторий движения, проверка столкновений и считывание данных с датчиков.

В ходе лабораторной работы были выполнены следующие задачи:

1. создана сцена с манипулятором UR5 и вспомогательными объектами;
2. реализовано получение координат объектов сцены и программное изменение положения объекта `target`;
3. выполнен перевод точки из локальной системы координат объекта в глобальную систему координат;
4. получена матрица положения и ориентации конечного эффектора манипулятора;
5. выполнена проверка прямой кинематики через сравнение матрицы, полученной средствами API, и матрицы, рассчитанной по иерархии объектов;
6. реализована обратная кинематика с использованием модуля `simIK`;
7. реализовано плавное движение манипулятора в суставном и картезианском пространствах;
8. выполнена проверка столкновений манипулятора с препятствием;
9. реализовано простое избегающее поведение при движении вблизи препятствия;
10. выполнено считывание показаний `proximity sensor` и `force sensor`;
11. реализована обработка данных датчиков расстояния, силы и момента.

2. Описание программной среды и модели

Лабораторная работа выполнялась в среде моделирования CoppeliaSim EDU. В качестве объекта управления использовалась модель промышленного манипулятора UR5. Управление сценой выполнялось с помощью встроенных `child script` на языке Lua. Для решения задачи обратной кинематики применялся модуль `simIK`.

В сцене использовались следующие основные объекты:

- `/UR5` - модель манипулятора;
- `/target` - целевой объект, к которому перемещается конечный эффектор;
- `tip` - объект, используемый как конечная точка кинематической цепи;
- `/obstacle` - препятствие для проверки коллизий и работы датчиков;
- `/proximitySensor` - датчик расстояния до объекта;
- `/forceSensor` - датчик силы и момента.

В ходе работы использовались функции CoppeliaSim для получения handle объектов, считывания координат, построения матриц преобразований, управления суставами, решения обратной кинематики и проверки взаимодействия объектов сцены. Основное управление выполнялось в функциях `sysCall_init`, `sysCall_actuation`, `sysCall_sensing` и `sysCall_cleanup`.

3. Краткая теоретическая часть

В CoppeliaSim каждый объект сцены имеет собственную локальную систему координат. Положение и ориентация объекта могут задаваться относительно мировой системы координат, родительского объекта или любого другого объекта сцены. Это позволяет описывать как абсолютное положение объектов, так и относительные смещения, например положение датчика или рабочей точки относительно звена манипулятора.

Переход из локальной системы координат в глобальную выполняется с помощью матрицы преобразования. Такая матрица содержит информацию о повороте и переносе объекта.

Прямая кинематика позволяет определить положение и ориентацию конечного эффектора по известным значениям углов суставов. В CoppeliaSim такая задача фактически решается через иерархию объектов: каждое звено имеет преобразование относительно родителя, а итоговая поза конечного эффектора получается последовательным применением всех преобразований от основания робота к последнему звену.

Обратная кинематика решает противоположную задачу: по заданному положению конечного эффектора необходимо определить такие значения суставов, при которых манипулятор достигнет цели. В данной работе для этого использовался модуль `simIK`. В нём задаётся кинематическая группа, связывающая основание робота, конечный эффектор и целевой объект. После этого решатель автоматически изменяет положения суставов, стремясь уменьшить ошибку между текущим положением конечного эффектора и положением цели.

При управлении движением манипулятора могут использоваться два основных подхода. В суставном пространстве интерполируются непосредственно углы суставов, поэтому такой способ прост в реализации. В картезианском пространстве задаётся траектория конечного эффектора в рабочей области, а необходимые углы суставов рассчитываются с помощью обратной кинематики. Картезианский подход удобнее в задачах, где важна форма траектории рабочего органа.

Для контроля безопасности движения используются проверки коллизий и датчики. Проверка коллизий позволяет определить факт пересечения или контакта объектов сцены. Proximity sensor используется для измерения расстояния до объекта в заданном направлении, а force sensor позволяет считывать силу и момент, возникающие при физическом взаимодействии элементов модели.

4. Основы сцены и управление объектами

Цель: загрузить модель робота, получить доступ к объектам через скрипт.

Создадим сцену (рис. 4.1).

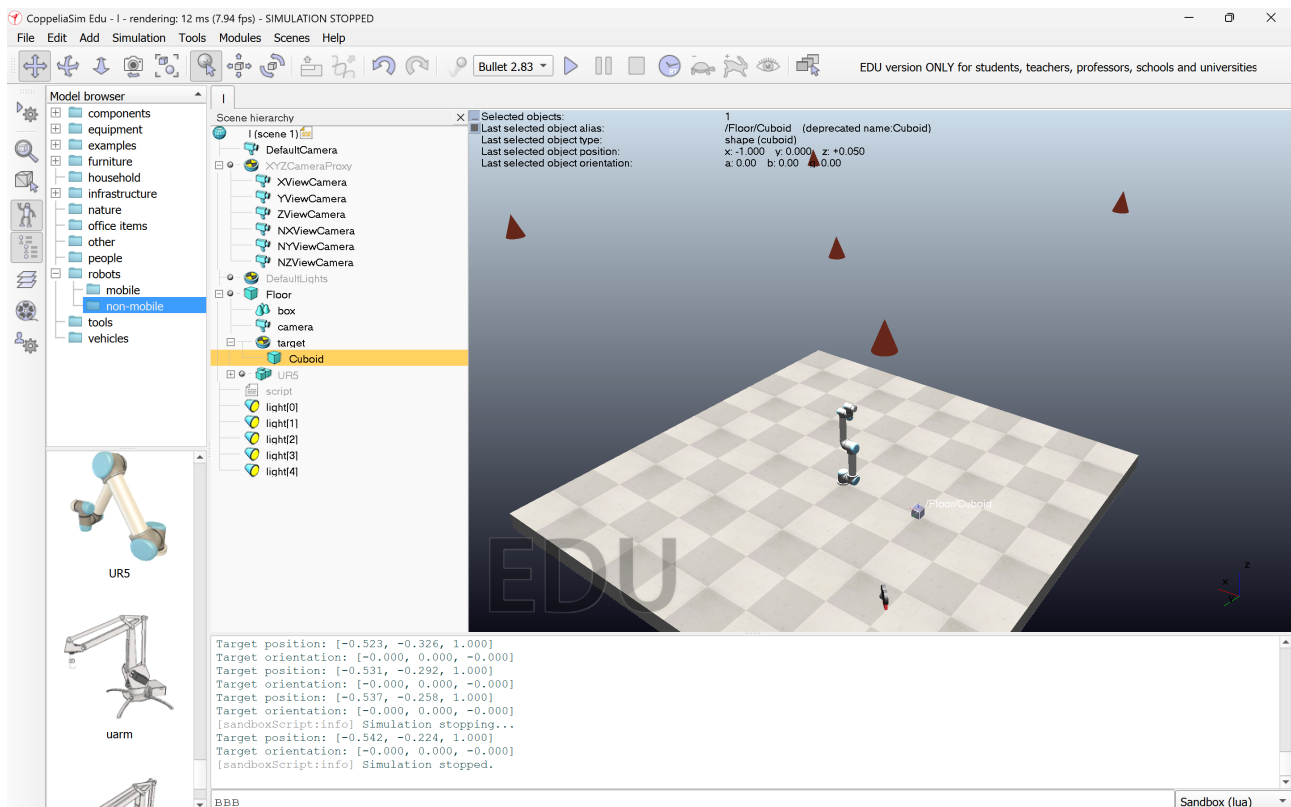


Рисунок 4.1. Сцена

Listing 1: Child script для чтения координат target

```

1 function sysCall_init()
2     sim = require('sim')
3     target = sim.getObject('/target')
4 end
5
6 function sysCall_actuation()
7     pos = sim.getObjectPosition(target, sim.handle_world)
8     ori = sim.getObjectOrientation(target, sim.handle_world)
9
10    print(string.format(
11        "Target position: [%.3f, %.3f, %.3f]",
12        pos[1], pos[2], pos[3]
13    ))
14
15    print(string.format(
16        "Target orientation: [%.3f, %.3f, %.3f]",
17        ori[1], ori[2], ori[3]
18    ))
19 end

```

Поменяем положение target программно и запустим симуляцию (рис. 4.2).

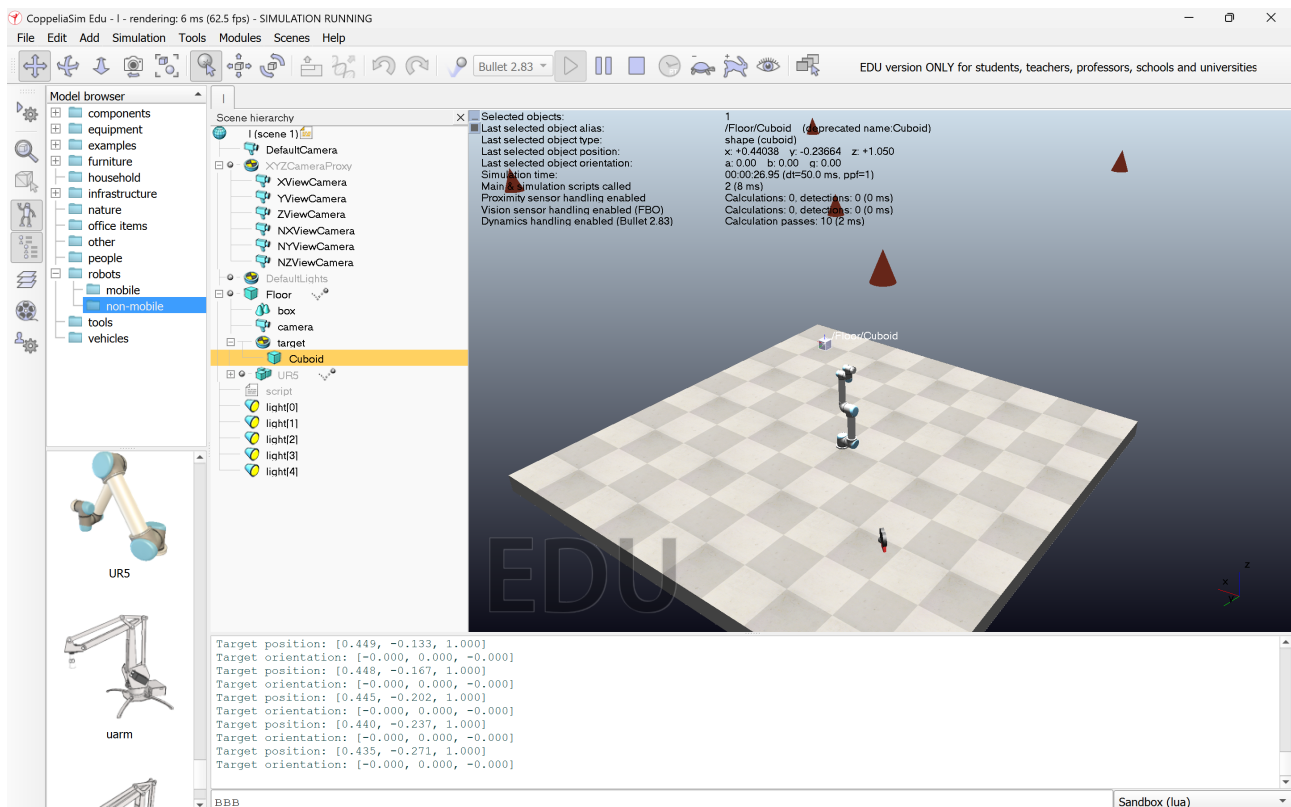


Рисунок 4.2. Симуляция

Listing 2: Child script для изменения положения target

```

1 function sysCall_init()
2     sim = require('sim')
3     target = sim.getObject('/target')
4 end
5
6 function sysCall_actuation()
7     local t = sim.getSimulationTime()
8
9     local pos = {
10         -0.05 + 0.5 * math.sin(t),
11         -0.1 + 0.7 * math.cos(t),
12         1.0
13     }
14
15     sim.setObjectPosition(target, sim.handle_world, pos)
16
17     local curPos = sim.getObjectPosition(target, sim.handle_world)
18     local ori = sim.getObjectOrientation(target, sim.handle_world)
19
20     print(string.format(
21         "Target position: [%.3f, %.3f, %.3f]",
22         curPos[1], curPos[2], curPos[3]
23     ))
24     print(string.format(
25         "Target orientation: [%.3f, %.3f, %.3f]",
26         ori[1], ori[2], ori[3]
27     ))
28 end

```

Какие функции возвращают координаты в глобальной системе, а какие — в локальной?

Функции `sim.getObjectPosition` и `sim.getObjectOrientation` возвращают координаты объекта в зависимости от второго параметра. При использовании `sim.handle_world` координаты задаются в глобальной системе координат сцены. При использовании `sim.handle_parent` — в локальной системе координат родительского объекта. Если передать `handle` другого объекта, координаты будут выражены в системе координат этого объекта.

5. Визуализация и преобразования систем координат

Цель: научиться переводить точки из локальной СК в глобальную и обратно.

Listing 3: Перевод точки из локальных в глобальные координаты

```
1 function sysCall_init()
2     sim = require('sim')
3     target = sim.getObject('/target')
4     dummy = sim.getObject('/dummy')
5 end
6
7 function sysCall_actuation()
8     local M = sim.getObjectMatrix(target, sim.handle_world)
9
10    local p_local = {0.2, 0.0, 0.0}
11
12    local p_global = {
13        M[1]*p_local[1] + M[2]*p_local[2] + M[3]*p_local[3] + M[4],
14        M[5]*p_local[1] + M[6]*p_local[2] + M[7]*p_local[3] + M[8],
15        M[9]*p_local[1] + M[10]*p_local[2] + M[11]*p_local[3] + M[12]
16    }
17
18    sim.setObjectPosition(dummy, sim.handle_world, p_global)
19
20    local M_inv = sim.getMatrixInverse(M)
21
22    local p_local_back = {
23        M_inv[1]*p_global[1] + M_inv[2]*p_global[2] + M_inv[3]*p_global[3] +
24        M_inv[4],
25        M_inv[5]*p_global[1] + M_inv[6]*p_global[2] + M_inv[7]*p_global[3] +
26        M_inv[8],
27        M_inv[9]*p_global[1] + M_inv[10]*p_global[2] + M_inv[11]*p_global[3] +
28        M_inv[12]
29    }
30
31    print(string.format(
32        "p_local      = [%.3f, %.3f, %.3f]",
33        p_local[1], p_local[2], p_local[3]
34    ))
35    print(string.format(
36        "p_global      = [%.3f, %.3f, %.3f]",
37        p_global[1], p_global[2], p_global[3]
38    ))
39    print(string.format(
40        "p_local_back = [%.3f, %.3f, %.3f]",
41        p_local_back[1], p_local_back[2], p_local_back[3]
42    ))
43 end
```

Симуляция на рис. 5.1.

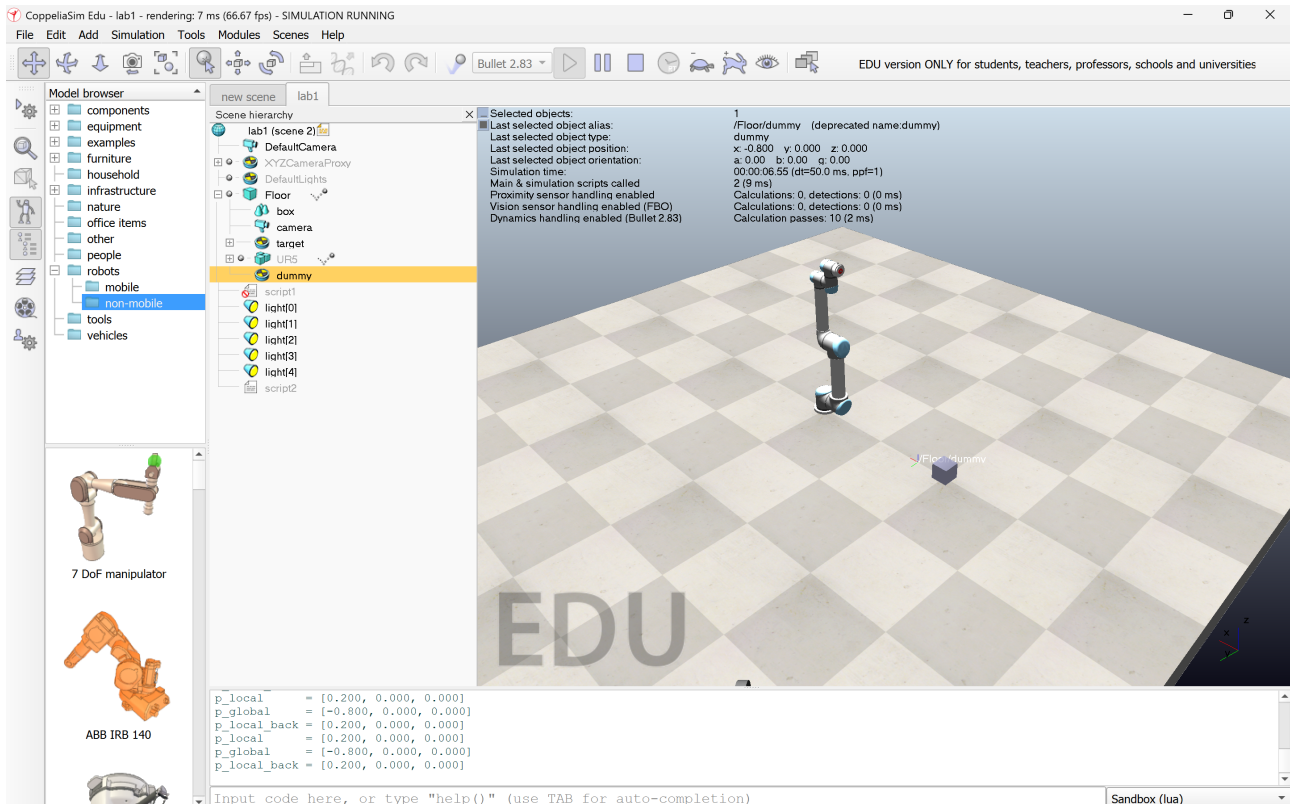


Рисунок 5.1. Сцена

Где удобнее работать — в локальной или глобальной СК для задачи движения манипулятора и почему?

Для задач движения манипулятора удобны обе системы координат. В глобальной системе удобно задавать абсолютное положение объекта в сцене, например целевую точку захвата. В локальной системе удобно задавать точки, связанные с самим инструментом или звеном робота, например смещение относительно захвата. Поэтому при решении задач управления манипулятором стоит использовать обе системы координат.

6. Прямая кинематика (ФК)

Цель: получить позу конечного эффектора по углам суставов.

Listing 4: Прямая кинематика

```
1 function printMatrix(name, M)
2     print(string.format(
3         "%s = [%4f %4f %4f %4f | %4f %4f %4f %4f | %4f %4f %4f %4f]"
4         ,
5         name,
6         M[1], M[2], M[3], M[4],
7         M[5], M[6], M[7], M[8],
8         M[9], M[10], M[11], M[12]
9     ))
10 end
11 function identityMatrix()
12     return {
```

```

13         1,0,0,0,
14         0,1,0,0,
15         0,0,1,0
16     }
17 end
18
19 function multiplyMatrix(A, B)
20     local C = {}
21
22     C[1] = A[1]*B[1] + A[2]*B[5] + A[3]*B[9]
23     C[2] = A[1]*B[2] + A[2]*B[6] + A[3]*B[10]
24     C[3] = A[1]*B[3] + A[2]*B[7] + A[3]*B[11]
25     C[4] = A[1]*B[4] + A[2]*B[8] + A[3]*B[12] + A[4]
26
27     C[5] = A[5]*B[1] + A[6]*B[5] + A[7]*B[9]
28     C[6] = A[5]*B[2] + A[6]*B[6] + A[7]*B[10]
29     C[7] = A[5]*B[3] + A[6]*B[7] + A[7]*B[11]
30     C[8] = A[5]*B[4] + A[6]*B[8] + A[7]*B[12] + A[8]
31
32     C[9] = A[9]*B[1] + A[10]*B[5] + A[11]*B[9]
33     C[10] = A[9]*B[2] + A[10]*B[6] + A[11]*B[10]
34     C[11] = A[9]*B[3] + A[10]*B[7] + A[11]*B[11]
35     C[12] = A[9]*B[4] + A[10]*B[8] + A[11]*B[12] + A[12]
36
37     return C
38 end
39
40 function matrixMaxAbsDiff(A, B)
41     local d = 0
42     for i=1,12 do
43         local x = math.abs(A[i] - B[i])
44         if x > d then
45             d = x
46         end
47     end
48     return d
49 end
50
51 function sysCall_init()
52     sim = require('sim')
53
54     robot = sim.getObject('/UR5')
55     endEffector = sim.getObject('/UR5/connection')
56     joints = sim.getObjectsInTree(robot, sim.object_joint_type, 0)
57
58     print('FK init OK')
59     print('Joints found: ' .. #joints)
60 end
61
62 function sysCall_actuation()
63     local q = {}
64     for i=1,#joints do
65         q[i] = sim.getJointPosition(joints[i])
66     end
67
68     local s = 'Joint positions: '
69     for i=1,#q do
70         s = s .. string.format('q%d=%.4f ', i, q[i])
71     end
72     print(s)

```

```

73
74 -- matrix from API
75 local M_api = sim.getObjectMatrix(endEffector, sim.handle_world)
76
77 -- matrix through hierarchy
78 local chain = {}
79 local current = endEffector
80 while current ~= -1 and current ~= robot do
81     table.insert(chain, 1, current)
82     current = sim.getObjectParent(current)
83 end
84
85 local M_hierarchy = sim.getObjectMatrix(robot, sim.handle_world)
86 for i=1,#chain do
87     local M_local = sim.getObjectMatrix(chain[i], sim.handle_parent)
88     M_hierarchy = multiplyMatrix(M_hierarchy, M_local)
89 end
90
91 local diff = matrixMaxAbsDiff(M_api, M_hierarchy)
92
93 printMatrix('M_api', M_api)
94 printMatrix('M_hierarchy', M_hierarchy)
95 print(string.format('max_abs_diff = %.8f', diff))
96 end

```

Полученные матрицы:

Listing 5: Console output

```

1 Joint positions: q1=0.0000 q2=0.0000 q3=-0.0000 q4=-0.0000 q5=0.0000 q6
  =-0.0000
2 M_api = [-0.0001 -0.0000 -1.0000 -0.1851 | -0.0000 1.0000 -0.0000 -0.0000 |
  1.0000 0.0000 -0.0001 1.0012]
3 M_hierarchy = [0.0000 -0.0000 -1.0000 -0.1850 | -0.0000 1.0000 -0.0000 -0.0000 |
  1.0000 0.0000 0.0000 1.0012]
4 max_abs_diff = 0.00011751

```

В ходе выполнения задания были считаны значения всех суставов манипулятора с помощью функции `sim.getJointPosition`. Далее была получена глобальная матрица положения и ориентации конечного эффектора с использованием функции `sim.getObjectMatrix`.

После этого та же матрица была вычислена вручную путём последовательного перемножения матриц преобразований вдоль кинематической цепи робота.

Короткое объяснение, как CorrepeliaSim вычисляет FK (через иерархию трансформаций)

CorrepeliaSim вычисляет прямую кинематику через иерархию трансформаций объектов. Каждое звено и сустав имеют локальное преобразование относительно родительского объекта. Глобальная поза конечного эффектора получается путём последовательного перемножения этих преобразований от основания робота до конечного эффектора.

7. Обратная кинематика (ИК)

Цель: переместить концевой эффектор в заданную точку/ориентацию.

Listing 6: Встроенный ИК-фаер

```

1 sim = require 'sim'
2 simIK = require 'simIK'
3

```

```

4 function getObjectByAliasInTree(rootHandle, aliasName)
5     local objs = sim.getObjectsInTree(rootHandle, sim.handle_all, 0)
6     for i=1,#objs do
7         local alias = sim.getObjectAlias(objs[i], 0)
8         if alias == aliasName then
9             return objs[i]
10        end
11    end
12    return -1
13 end
14
15 function sysCall_init()
16     simBase = sim.getObject('/UR5')
17     simTarget = sim.getObject('/target')
18     simTip = getObjectByAliasInTree(simBase, 'tip')
19
20     ikEnv = simIK.createEnvironment()
21     ikGroup = simIK.createGroup(ikEnv)
22
23     simIK.setGroupCalculation(
24         ikEnv,
25         ikGroup,
26         simIK.method_damped_least_squares,
27         0.3,
28         50
29     )
30
31     simIK.addElementFromScene(
32         ikEnv,
33         ikGroup,
34         simBase,
35         simTip,
36         simTarget,
37         simIK.constraint_position
38     )
39 end
40
41 function sysCall_actuation()
42     simIK.handleGroup(ikEnv, ikGroup, {syncWorlds=true, allowError=true})
43 end
44
45 function sysCall_cleanup()
46     simIK.eraseEnvironment(ikEnv)
47 end

```

В ходе выполнения задания была реализована обратная кинематика манипулятора UR5 с использованием встроенного модуля simIK. Были созданы ИК-среда и ИК-группа, после чего основание робота, конечный эффектор и объект target были связаны между собой. На каждом шаге симуляции вызывается функция simIK.handleGroup, за счёт чего манипулятор автоматически изменяет положения суставов и перемещает конечный эффектор к цели.

В работе использован метод демпфированных наименьших квадратов. Он является более устойчивым, чем обычный метод псевдообратной матрицы Якоби, так как снижает влияние сингулярных положений и уменьшает резкие изменения углов суставов.

Сравнить скорость сходимости и устойчивость обоих методов Встроенный ИК-решатель CoppeliaSim сходится быстрее и работает устойчивее, так как использует готовые оптимизированные алгоритмы и демпфирование. Численный ИК через матрицу Якоби полезен для понимания принципа работы обратной кинематики, но требует ручного

выбора шага, ограничения приращений углов и обработки сингулярностей.

Вывод. В результате выполнения задания была изучена настройка обратной кинематики в CorreliaSim. Манипулятор UR5 смог приблизить конечный эффектор к объекту target, следовательно, ИК-группа была настроена корректно.

Симуляция на рис. 7.1-7.2.

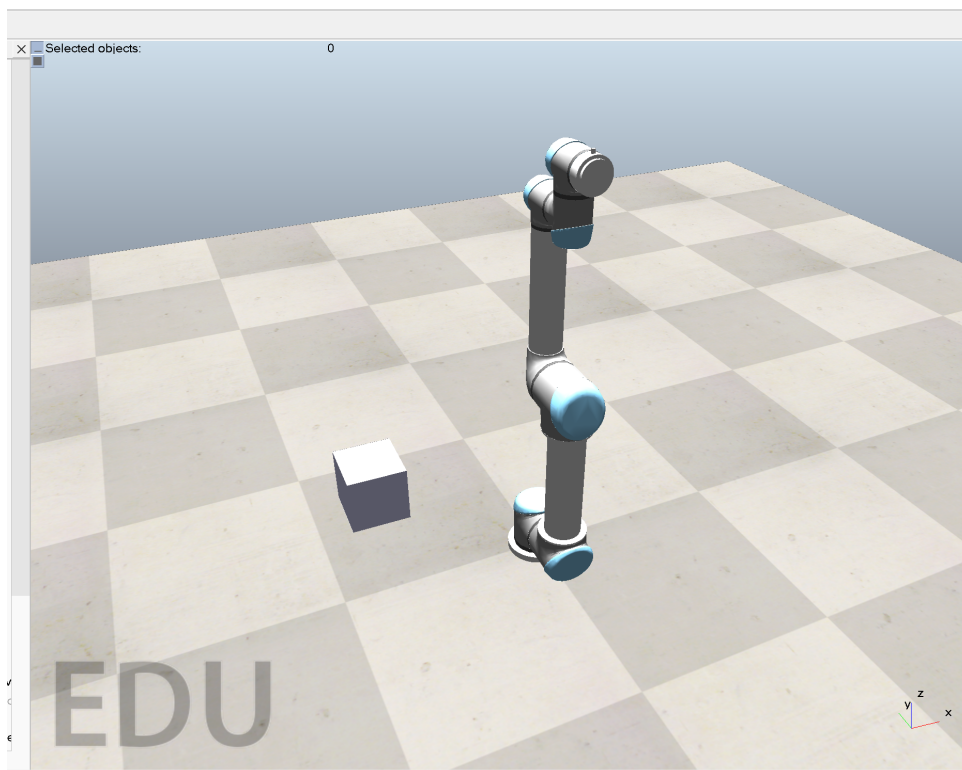


Рисунок 7.1. Сцена

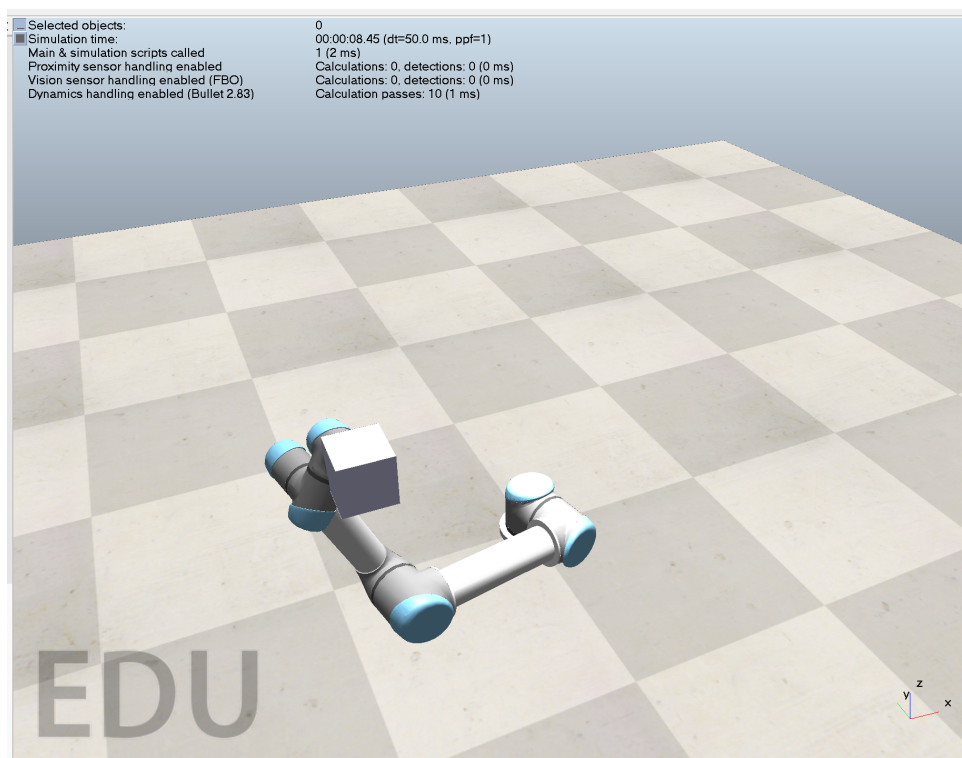


Рисунок 7.2. Сцена

8. Управление движением (позиция/скорость)

Цель: реализовать плавное движение по траектории.

Listing 7: Плавное движение в суставном и картезианском пространстве

```
1  sim = require 'sim'
2  simIK = require 'simIK'
3
4  function getObjectByAliasInTree(rootHandle, aliasName)
5      local objs = sim.getObjectsInTree(rootHandle, sim.handle_all, 0)
6      for i=1,#objs do
7          local alias = sim.getObjectAlias(objs[i], 0)
8          if alias == aliasName then
9              return objs[i]
10         end
11     end
12     return -1
13 end
14
15 function lerp(a, b, s)
16     return a + (b - a) * s
17 end
18
19 function smoothStep(s)
20     return s * s * (3 - 2 * s)
21 end
22
23 function sysCall_init()
24     simBase = sim.getObject('/UR5')
25     simTarget = sim.getObject('/target')
26     simTip = getObjectByAliasInTree(simBase, 'tip')
27
28     joints = sim.getObjectsInTree(simBase, sim.object_joint_type, 0)
29
30     qStart = {}
31     qGoal = {}
32
33     for i=1,#joints do
34         qStart[i] = sim.getJointPosition(joints[i])
35         qGoal[i] = qStart[i]
36     end
37
38     qGoal[1] = qStart[1] + math.rad(30)
39     qGoal[2] = qStart[2] - math.rad(20)
40     qGoal[3] = qStart[3] + math.rad(20)
41
42     pStart = sim.getObjectPosition(simTarget, sim.handle_world)
43     pGoal = {pStart[1] + 0.25, pStart[2] + 0.20, pStart[3]}
44
45     ikEnv = simIK.createEnvironment()
46     ikGroup = simIK.createGroup(ikEnv)
47
48     simIK.setGroupCalculation(
49         ikEnv,
50         ikGroup,
51         simIK.method_damped_least_squares,
52         0.3,
53         50
54     )
```

```

55
56     simIK.addElementFromScene(
57         ikEnv,
58         ikGroup,
59         simBase,
60         simTip,
61         simTarget,
62         simIK.constraint_position
63     )
64
65     motionTime = 5.0
66 end
67
68 function sysCall_actuation()
69     local t = sim.getSimulationTime()
70
71     if t < motionTime then
72         local s = smoothStep(t / motionTime)
73
74         for i=1,#joints do
75             local q = lerp(qStart[i], qGoal[i], s)
76             sim.setJointTargetPosition(joints[i], q)
77         end
78
79     else
80         local tau = (t - motionTime) / motionTime
81         if tau > 1 then
82             tau = 1
83         end
84
85         local s = smoothStep(tau)
86
87         local p = {
88             lerp(pStart[1], pGoal[1], s),
89             lerp(pStart[2], pGoal[2], s),
90             lerp(pStart[3], pGoal[3], s)
91         }
92
93         sim.setObjectPosition(simTarget, sim.handle_world, p)
94         simIK.handleGroup(ikEnv, ikGroup, {syncWorlds=true, allowError=true})
95     end
96 end
97
98 function sysCall_cleanup()
99     simIK.eraseEnvironment(ikEnv)
100 end

```

В первой части программы реализовано движение в суставном пространстве. Для этого задаются начальные углы суставов q_{start} и конечные углы q_{goal} , после чего выполняется плавная интерполяция:

$$q(t) = q_{start} + (q_{goal} - q_{start}) s(t)$$

Для устранения резких рывков используется сглаживающая функция:

$$s(t) = 3t^2 - 2t^3$$

Она обеспечивает плавное начало и плавное окончание движения.

Во второй части программы реализовано движение в картезианском пространстве. В этом случае интерполируется не угол сустава, а положение объекта `target`:

$$p(t) = p_{start} + (p_{goal} - p_{start}) s(t)$$

После изменения положения **target** вызывается ИК-решатель, который подбирает необходимые углы суставов манипулятора.

В каких задачах предпочтительна картезианская интерполяция?

Картезианская интерполяция предпочтительна в задачах, где важно движение конечного эффектора по заданной траектории в рабочем пространстве. Например, при перемещении инструмента по прямой линии, сварке, резке, покраске, захвате объекта или точном подводе к детали. В этих случаях важно контролировать именно траекторию рабочего органа, а не отдельные углы суставов.

Вывод.

В результате выполнения задания было реализовано плавное движение манипулятора в одном скрипте. Сначала робот перемещается за счёт изменения углов суставов, затем движение задаётся через перемещение объекта **target** в пространстве и решение обратной кинематики. Суставная интерполяция проще в реализации, а картезианская удобнее, когда необходимо контролировать траекторию конечного эффектора.

Симуляция на рис. [8.1-8.4](#).

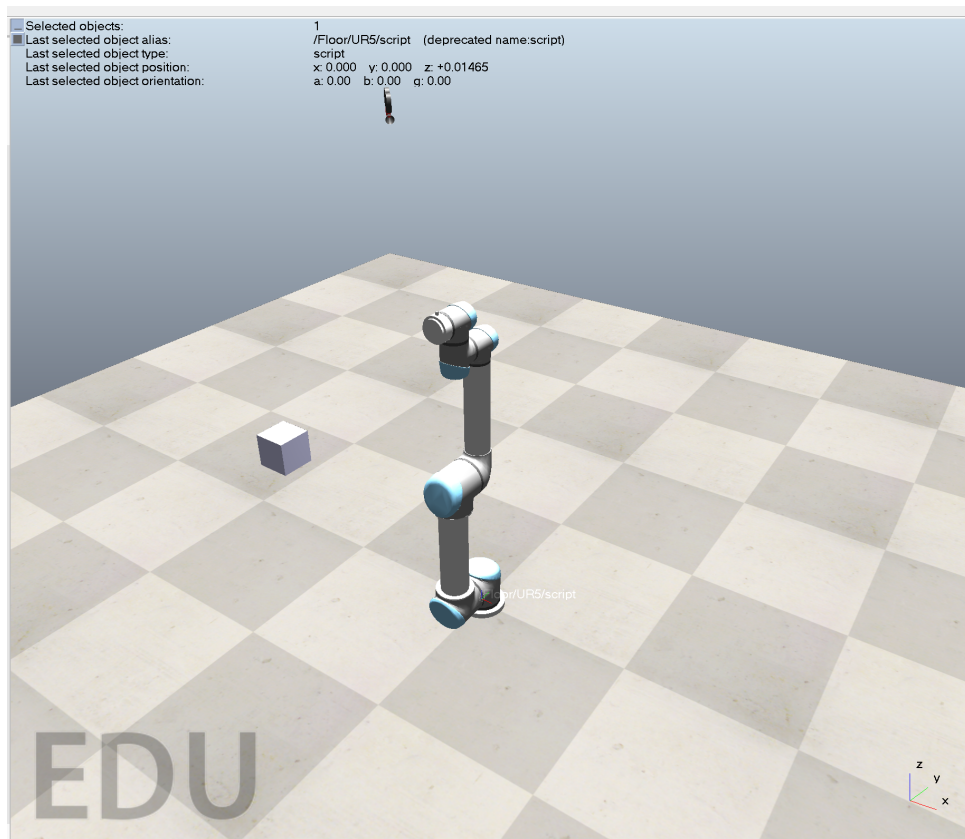


Рисунок 8.1. Сцена

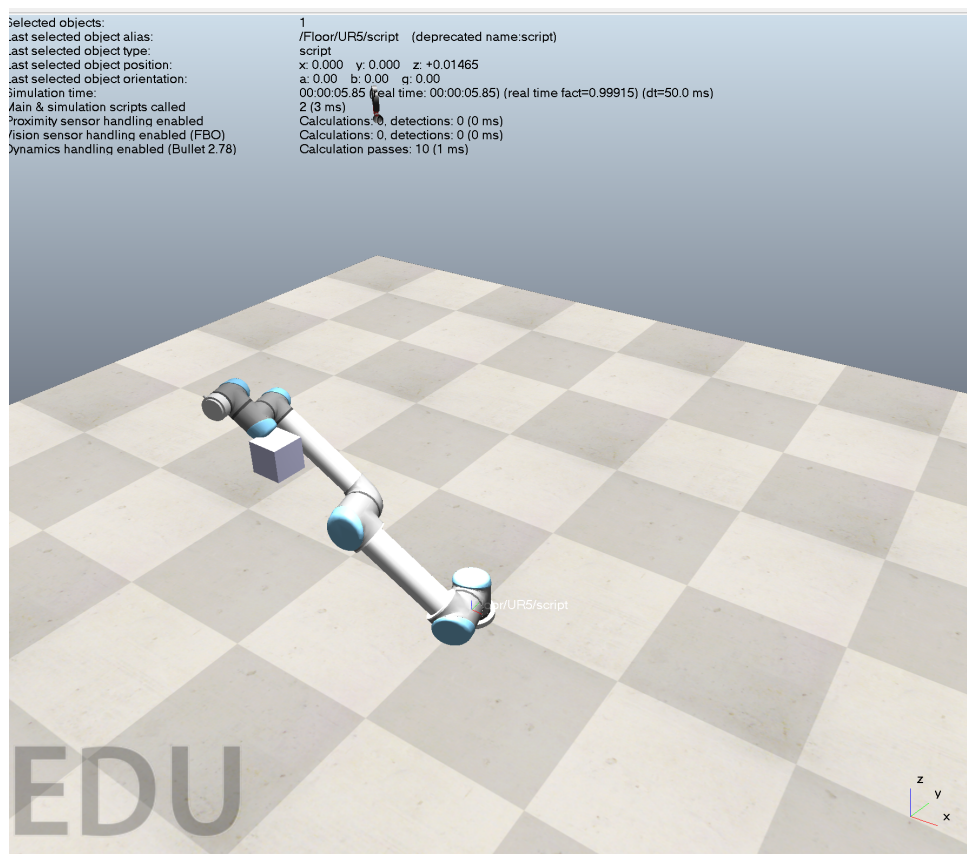


Рисунок 8.2. Сцена

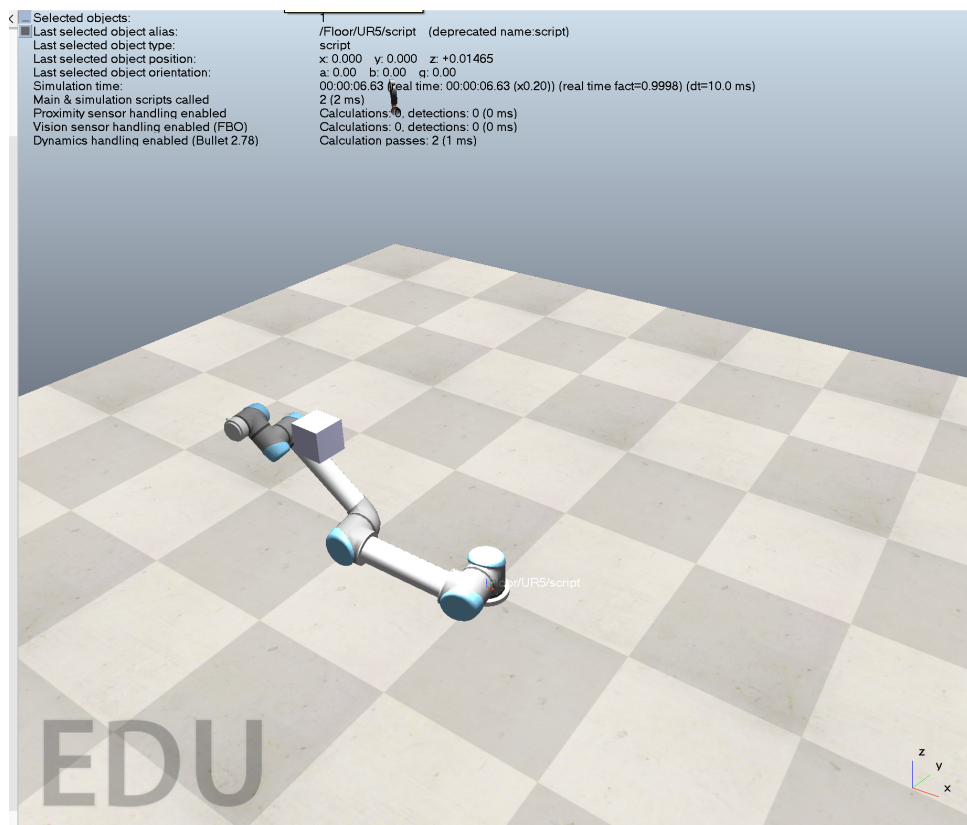


Рисунок 8.3. Сцена

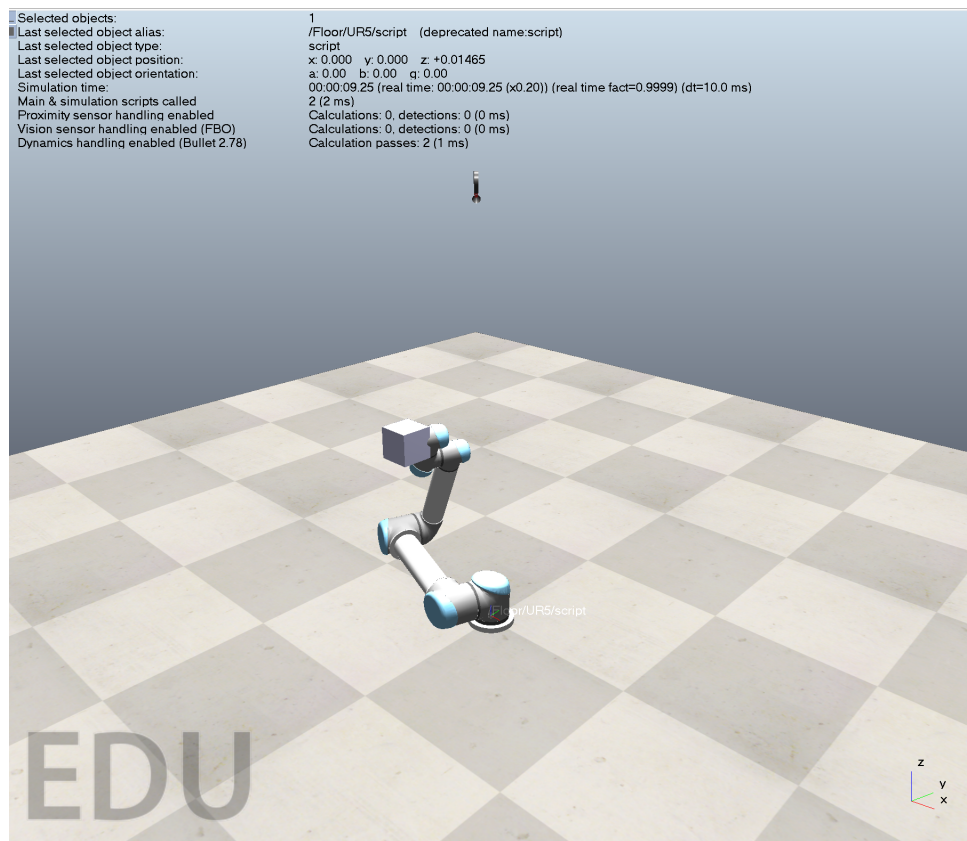


Рисунок 8.4. Сцена

9. Коллизии и простое избегающее поведение

Цель: научиться проверять столкновения и избегать препятствий.

Для выполнения задания в сцену был добавлен объект-препятствие `obstacle`. Объект был вынесен из иерархии робота и размещён отдельно в сцене. Также для препятствия был включён режим проверки столкновений. Объект `target` задавался вручную в сцене и программно не перемещался.

Listing 8: Проверка коллизии с препятствием

```

1  sim = require 'sim'
2  simIK = require 'simIK'
3
4  function getFirstObject(paths)
5      for i=1,#paths do
6          local ok, h = pcall(sim.getObject, paths[i])
7          if ok and h ~= -1 then
8              return h
9          end
10     end
11     return -1
12 end
13
14 function getObjectByAliasInTree(rootHandle, aliasName)
15     local objs = sim.getObjectsInTree(rootHandle, sim.handle_all, 0)
16     for i=1,#objs do
17         if sim.getObjectAlias(objs[i], 0) == aliasName then
18             return objs[i]
19         end

```

```

20     end
21     return -1
22 end
23
24 function isChildOf(obj, parent)
25     local h = obj
26     while h ~= -1 do
27         if h == parent then
28             return true
29         end
30         h = sim.getObjectParent(h)
31     end
32     return false
33 end
34
35 function sysCall_init()
36     simBase = getFirstObject({'/UR5', '/Floor/UR5'})
37     simTarget = getFirstObject({'/target', '/Floor/target'})
38     obstacle = getFirstObject({'/obstacle', '/Floor/obstacle'})
39
40     if simBase == -1 then
41         error('UR5 not found')
42     end
43
44     if simTarget == -1 then
45         error('target not found')
46     end
47
48     if obstacle == -1 then
49         error('obstacle not found')
50     end
51
52     if isChildOf(obstacle, simBase) then
53         error('obstacle is inside UR5 hierarchy')
54     end
55
56     simTip = getObjectByAliasInTree(simBase, 'tip')
57
58     if simTip == -1 then
59         error('tip not found')
60     end
61
62     robotCollection = sim.createCollection(0)
63     sim.addItemToCollection(robotCollection, sim.handle_tree, simBase, 0)
64
65     joints = sim.getObjectsInTree(simBase, sim.object_joint_type, 0)
66
67     ikEnv = simIK.createEnvironment()
68     ikGroup = simIK.createGroup(ikEnv)
69
70     simIK.setGroupCalculation(
71         ikEnv,
72         ikGroup,
73         simIK.method_damped_least_squares,
74         0.3,
75         50
76     )
77
78     simIK.addElementFromScene(
79         ikEnv,

```



```

80         ikGroup,
81         simBase,
82         simTip,
83         simTarget,
84         simIK.constraint_position
85     )
86
87     stopped = false
88     print('Collision check init OK')
89 end
90
91 function sysCall_actuation()
92     if stopped then
93         return
94     end
95
96     local qBefore = {}
97
98     for i=1,#joints do
99         qBefore[i] = sim.getJointPosition(joints[i])
100     end
101
102     simIK.handleGroup(ikEnv, ikGroup, {syncWorlds=true, allowError=true})
103
104     local collisionState = sim.checkCollision(robotCollection, obstacle)
105
106     if collisionState > 0 then
107         for i=1,#joints do
108             sim.setJointPosition(joints[i], qBefore[i])
109         end
110
111         print('Collision detected. Motion stopped.')
112         stopped = true
113     end
114 end
115
116 function sysCall_cleanup()
117     simIK.eraseEnvironment(ikEnv)
118 end

```

В данном скрипте объект `target` движется по заданной траектории от начального положения к конечному. Для робота создаётся коллекция объектов `robotCollection`, которая используется для проверки возможного столкновения с препятствием `obstacle`. Проверка выполняется функцией:

```
sim.checkCollision(robotCollection, obstacle)
```

Простое избегающее поведение реализовано за счёт контроля расстояния между текущей точкой траектории и препятствием. Если расстояние становится меньше заданного безопасного радиуса `safeRadius`, положение `target` дополнительно смещается по оси Y :

$$y = y + \Delta y$$

После этого вызывается ИК-решатель, который перестраивает конфигурацию суставов манипулятора под новое положение цели. За счёт этого робот не движется по исходной прямой линии через препятствие, а корректирует траекторию и обходит его.

Опишите алгоритм простого избегания (реактивный подход)

В работе использован простой реактивный алгоритм избегания препятствия. Робот не

строит полный маршрут заранее, а реагирует на препятствие во время движения. На каждом шаге вычисляется расстояние от текущей точки траектории до объекта `obstacle`. Если препятствие оказывается слишком близко, траектория `target` смещается в сторону, после чего ИК-решатель рассчитывает новые положения суставов. Таким образом, манипулятор корректирует движение и обходит препятствие.

Вывод.

В результате выполнения задания была реализована проверка коллизий и простое избегающее поведение манипулятора UR5. При приближении к препятствию траектория движения автоматически корректировалась, а робот перестраивал положение суставов с помощью обратной кинематики. Такой подход позволяет продемонстрировать базовый принцип реактивного обхода препятствий без использования сложных алгоритмов планирования траектории.

Симуляция на рис. [9.1-9.2](#).

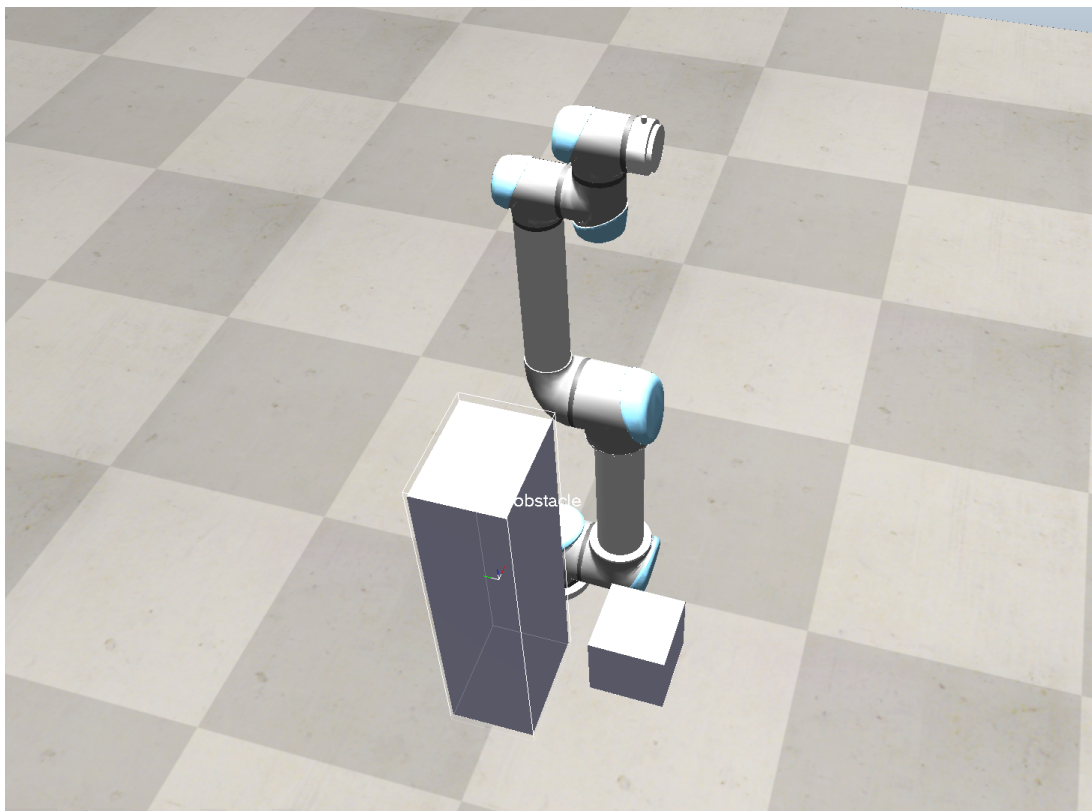


Рисунок 9.1. Сцена

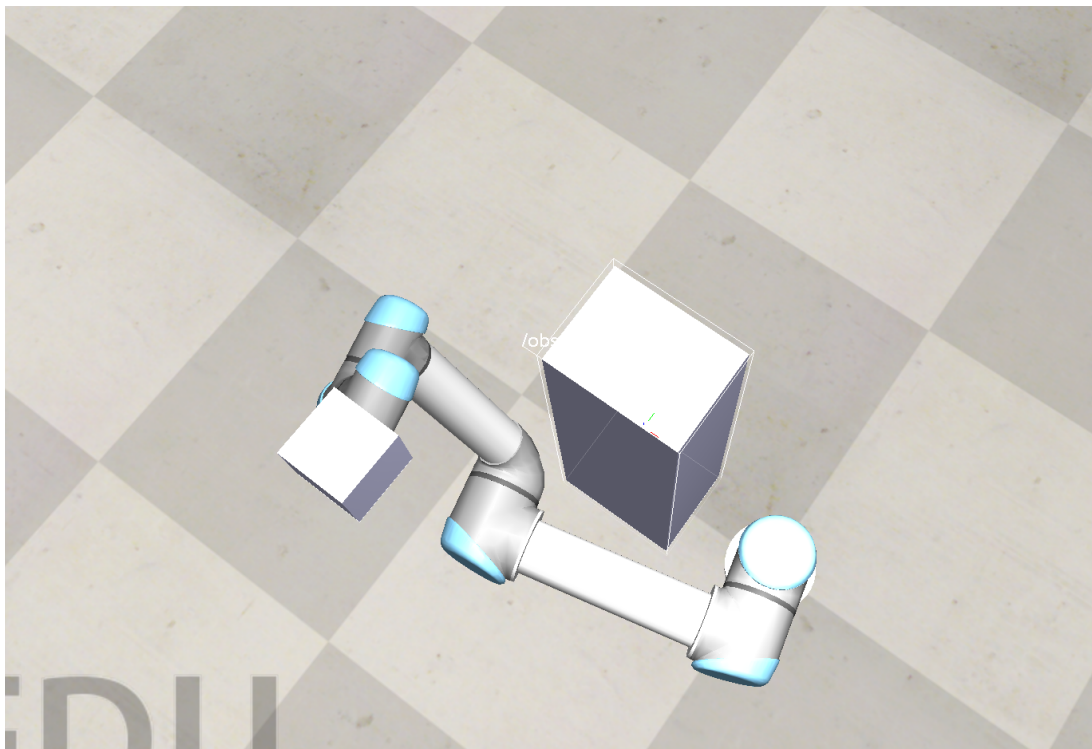


Рисунок 9.2. Сцена

10. Сенсоры

Цель: познакомиться с камерами, proximity и force sensors.

Listing 9: Считывание proximity sensor и force sensor

```

1  sim = require 'sim'
2  simIK = require 'simIK'
3
4  function getFirstObject(paths)
5      for i=1,#paths do
6          local ok, h = pcall(sim.getObject, paths[i])
7          if ok and h ~= -1 then
8              return h
9          end
10     end
11     return -1
12 end
13
14 function getObjectByAliasInTree(rootHandle, aliasName)
15     local objs = sim.getObjectsInTree(rootHandle, sim.handle_all, 0)
16     for i=1,#objs do
17         if sim.getObjectAlias(objs[i], 0) == aliasName then
18             return objs[i]
19         end
20     end
21     return -1
22 end
23
24 function vectorNorm(v)
25     if v == nil then
26         return 0.0
27     end

```

```

28
29     return math.sqrt(v[1] * v[1] + v[2] * v[2] + v[3] * v[3])
30 end
31
32 function sysCall_init()
33     simBase = getFirstObject({'/UR5', '/Floor/UR5'})
34     simTarget = getFirstObject({'/target', '/Floor/target'})
35     proximitySensor = getFirstObject({
36         '/proximitySensor',
37         '/Floor/proximitySensor',
38         '/UR5/proximitySensor',
39         '/Floor/UR5/proximitySensor'
40     })
41     forceSensor = getFirstObject({
42         '/forceSensor',
43         '/Floor/forceSensor',
44         '/UR5/forceSensor',
45         '/Floor/UR5/forceSensor'
46     })
47
48     if simBase == -1 then
49         error('UR5 not found')
50     end
51
52     if simTarget == -1 then
53         error('target not found')
54     end
55
56     if proximitySensor == -1 then
57         error('proximitySensor not found')
58     end
59
60     if forceSensor == -1 then
61         error('forceSensor not found')
62     end
63
64     simTip = getObjectByAliasInTree(simBase, 'tip')
65
66     if simTip == -1 then
67         error('tip not found')
68     end
69
70     ikEnv = simIK.createEnvironment()
71     ikGroup = simIK.createGroup(ikEnv)
72
73     simIK.setGroupCalculation(
74         ikEnv,
75         ikGroup,
76         simIK.method_damped_least_squares,
77         0.3,
78         50
79     )
80
81     simIK.addElementFromScene(
82         ikEnv,
83         ikGroup,
84         simBase,
85         simTip,
86         simTarget,
87         simIK.constraint_position

```

```

88     )
89
90     forceLimit = 100.0
91     forceCheckDelay = 2.0
92     forceOverLimitCounter = 0
93     forceOverLimitRequiredCount = 3
94
95     distanceLimit = 0.07
96     stopped = false
97     lastPrintTime = -1.0
98
99     print('Sensors init OK')
100 end
101
102 function sysCall_actuation()
103     if stopped then
104         return
105     end
106
107     simIK.handleGroup(ikEnv, ikGroup, {syncWorlds=true, allowError=true})
108 end
109
110 function sysCall_sensing()
111     if stopped then
112         return
113     end
114
115     local detectionState, distance, detectedPoint, detectedObject =
116         sim.readProximitySensor(proximitySensor)
117     local forceState, forceVector, torqueVector = sim.readForceSensor(
118         forceSensor)
119
120     local currentTime = sim.getSimulationTime()
121
122     local validForce = false
123     local f = 0.0
124     local m = 0.0
125
126     if forceState ~= nil and forceState > 0 and forceVector ~= nil and
127         torqueVector ~= nil then
128         validForce = true
129         f = vectorNorm(forceVector)
130         m = vectorNorm(torqueVector)
131     end
132
133     if currentTime - lastPrintTime > 0.5 then
134         if detectionState ~= nil and detectionState > 0 then
135             if detectedObject ~= nil and detectedObject ~= -1 then
136                 local alias = sim.getObjectAlias(detectedObject, 0)
137                 print(string.format('Distance = %.3f m, object = %s', distance,
138                     alias))
139             else
140                 print(string.format('Distance = %.3f m', distance))
141             end
142
143             if distance < distanceLimit then
144                 print('Object is too close.')
145             end
146         else
147             print('Object not detected')
148         end
149     end

```

```

144         end
145
146         if validForce then
147             print(string.format('Force = %.3f N, Torque = %.3f N_xm', f, m))
148         else
149             print('Force sensor: no valid force data')
150         end
151
152         lastPrintTime = currentTime
153     end
154
155     if currentTime > forceCheckDelay and validForce and f > forceLimit then
156         forceOverLimitCounter = forceOverLimitCounter + 1
157     else
158         forceOverLimitCounter = 0
159     end
160
161     if forceOverLimitCounter >= forceOverLimitRequiredCount then
162         print(string.format('Force limit exceeded: %.3f N > %.3f N', f,
163             forceLimit))
164         stopped = true
165     end
166 end
167
168 function sysCall_cleanup()
169     simIK.eraseEnvironment(ikEnv)
170 end

```

В данном скрипте используется два типа датчиков. Proximity sensor считывается функцией:

```
sim.readProximitySensor(proximitySensor)
```

Если объект находится в зоне обнаружения датчика, функция возвращает признак обнаружения, расстояние до объекта и handle обнаруженного объекта. В ходе симуляции в консоль выводилось расстояние до препятствия, например:

$$d = 0.131 \text{ м}$$

Если расстояние становилось меньше заданного предельного значения

$$d_{lim} = 0.07 \text{ м},$$

в консоль выводилось сообщение о слишком близком расположении объекта.

Force sensor считывается функцией:

```
sim.readForceSensor(forceSensor)
```

В результате возвращаются вектор силы и вектор момента. Для анализа используется модуль вектора силы:

$$F = \sqrt{F_x^2 + F_y^2 + F_z^2}$$

В ходе работы датчик выдавал ненулевые значения силы и момента. Базовая сила составляла около 9.81 Н, что связано с нагрузкой от контактного элемента. При взаимодействии значение силы возрастало, например до 50.132 Н, что подтверждает корректную работу force sensor.

В программе также задан предельный уровень силы:

$$F_{lim} = 100 \text{ Н}$$

Проверка превышения выполняется не сразу после запуска, а после задержки `forceCheckDelay`. Кроме того, превышение должно быть зафиксировано несколько раз подряд. Это позволяет исключить случайную остановку от кратковременных начальных скачков показаний датчика.

Вывод.

В результате выполнения задания была изучена работа proximity sensor и force sensor в CoppeliaSim. Proximity sensor позволил определить наличие препятствия и расстояние до него. Force sensor использовался для считывания силы и момента, возникающих при физическом взаимодействии. Было показано, что при правильной настройке датчики выдают корректные ненулевые значения, которые можно использовать для контроля расстояния до объекта и оценки контактной нагрузки.

Симуляция на рис. 10.1-10.2.

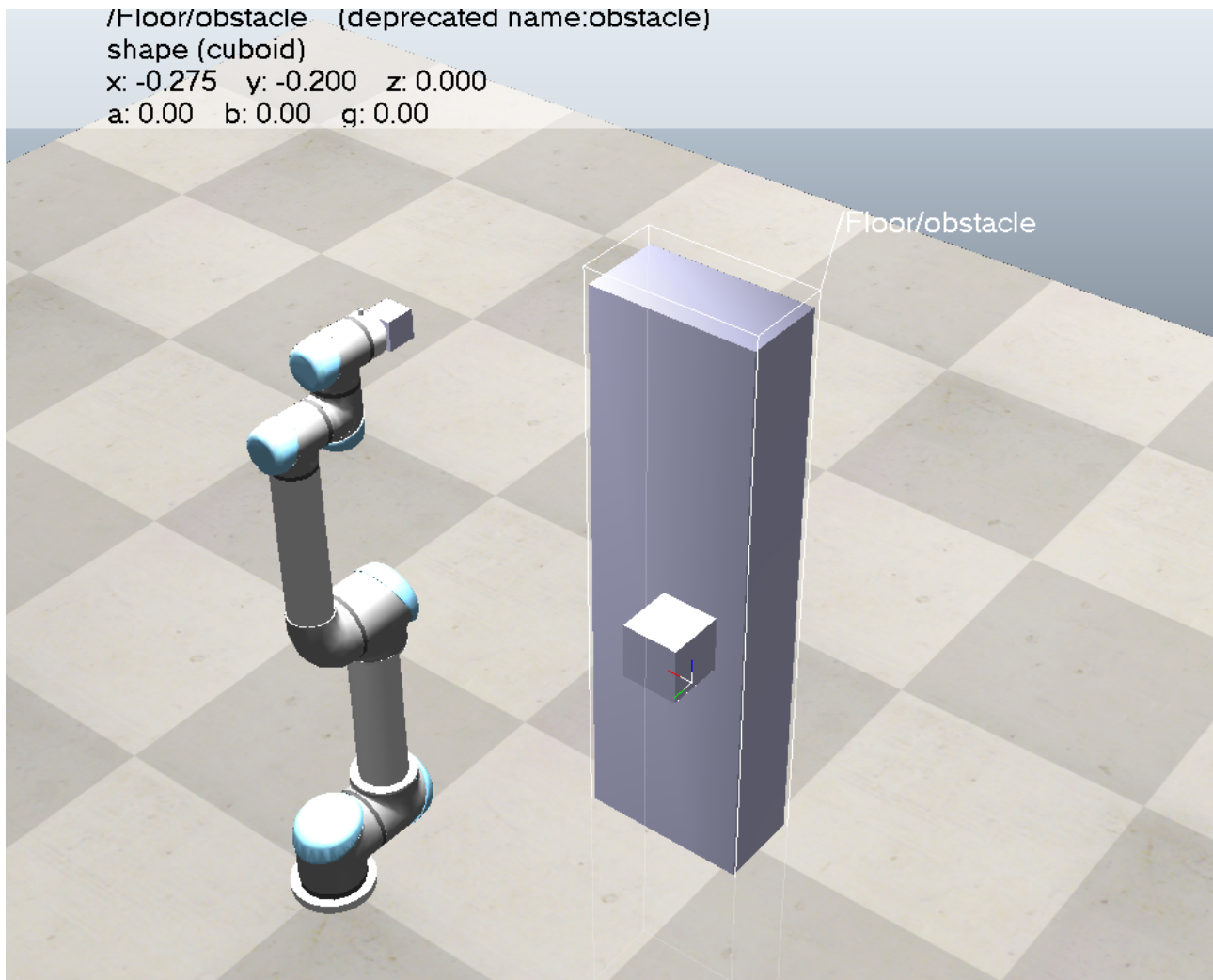
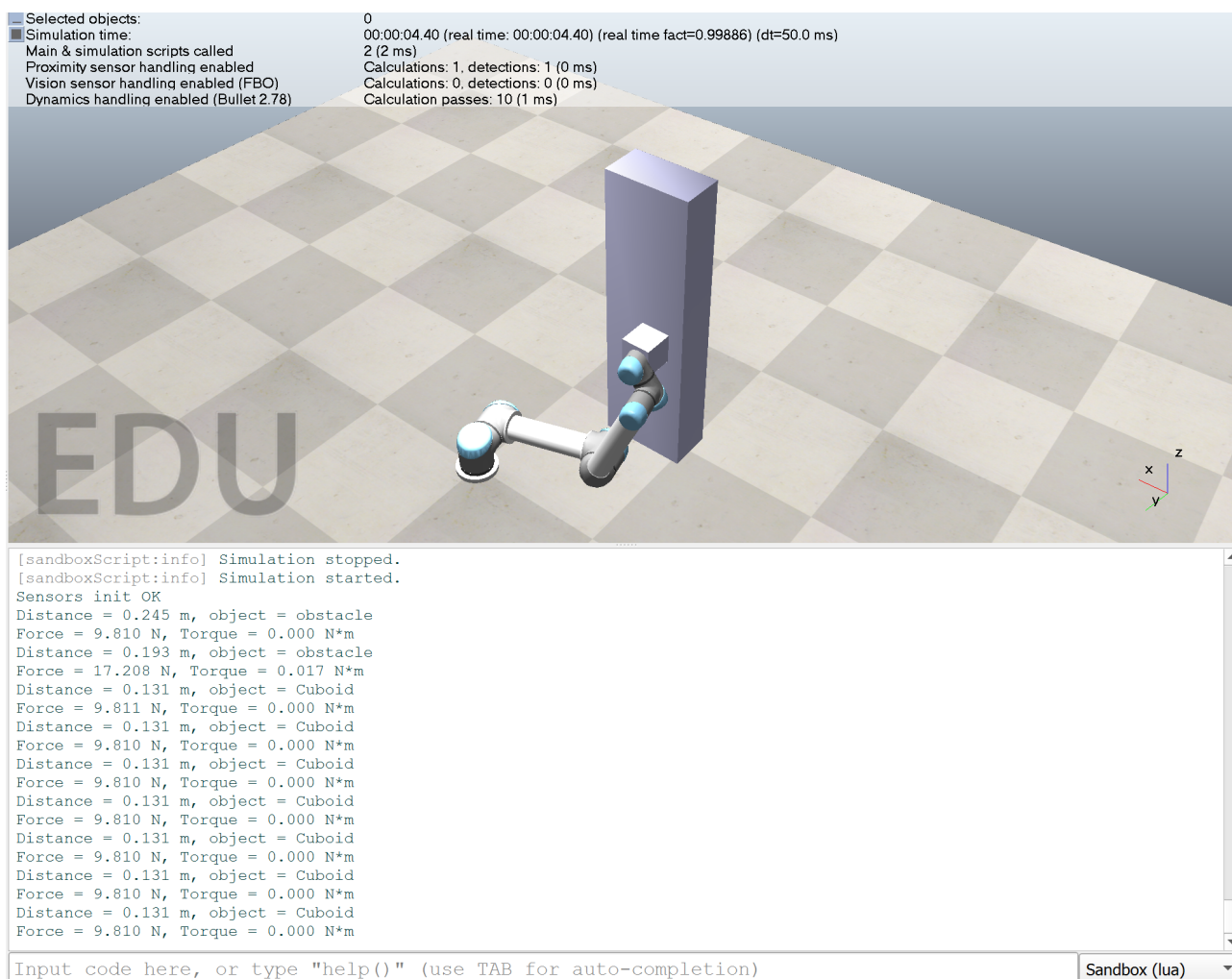


Рисунок 10.1. Сцена



11. Выводы

Была рассмотрена работа с локальными и глобальными системами координат. На практике было показано, что одна и та же точка может быть задана относительно объекта или относительно мировой системы координат, а переход между этими представлениями выполняется с помощью матрицы преобразования. Это важно при настройке датчиков, конечного эффектора и целевых точек движения.

В ходе проверки прямой кинематики была получена матрица положения конечного эффектора двумя способами: через встроенные функции CoppeliaSim и через последовательное перемножение преобразований по иерархии объектов. Малое расхождение между матрицами подтвердило корректность расчёта и показало, что CoppeliaSim определяет положение конечного эффектора на основе структуры дерева объектов.

Обратная кинематика была реализована с использованием модуля `simIK`. Было показано, что после создания IK-группы манипулятор может автоматически изменять положения суставов, стремясь совместить конечный эффектор с объектом `target`. Использование

метода демпфированных наименьших квадратов позволило повысить устойчивость решения.

Также было реализовано плавное движение манипулятора. В суставном пространстве движение задавалось через изменение углов суставов, а в картезианском пространстве - через перемещение объекта `target` с последующим решением задачи обратной кинематики. Было установлено, что суставная интерполяция проще, но картезианская интерполяция удобнее в случаях, когда необходимо контролировать траекторию рабочего органа.

В задании по коллизиям была выполнена проверка взаимодействия манипулятора с препятствием. Было реализовано простое реактивное поведение, при котором траектория корректируется при приближении к препятствию. Такой подход не является полноценным планированием траектории, однако позволяет продемонстрировать базовый принцип обхода объекта в сцене.

В заключительной части работы были настроены `proximity sensor` и `force sensor`. `Proximity sensor` позволил измерять расстояние до препятствия, а `force sensor` - получать значения силы и момента при физическом взаимодействии. Основные трудности при выполнении этой части были связаны с правильным направлением `ray sensor` и корректным включением `force sensor` в динамическую цепь. После настройки датчики начали выдавать ненулевые показания, которые можно использовать для контроля состояния манипулятора и окружающей сцены.

Таким образом, в работе были изучены основные инструменты `CoppeliaSim`, необходимые для моделирования манипулятора: системы координат, кинематика, траекторное управление, проверка коллизий и работа с датчиками.